DREW

Lecture 31: I/O and File in C

CSCI 260 Computer Systems & Architecture
Fall 2024

Andy Siyuan Feng

11/13/2024

Standard I/O in C

- *#include <stdio.h>* provides capabilities of using the following I/O functions:

  Formatted version

  - *printf()*
  - *scanf()*

  Non-formatted version: similar to those in LC-3

  - *putchar()*
  - *puts()*
  - *gets(): NOT safe, buffer overflow attack*

Standard I/O in C: *printf()*

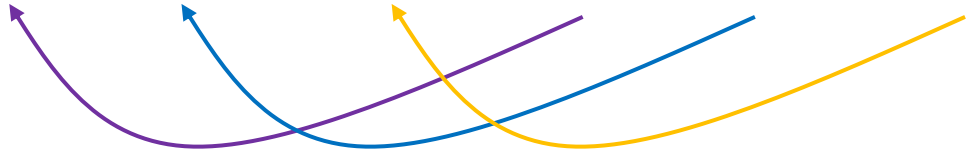
Printing variables

- In Java, mixing strings with variables to print:

```
System.out.println( “res = a x b = ” + …
                                num_a + “ x ” + num_b + “= ” + res + “.” );
```

- In C, specifiers are used (in fact Java also has this mode):

```
printf(“res = a x b = %d x %d = %d.”, num_a, num_b, res);
```



Standard I/O in C: *printf()*

Printing variables

• **Integers**:	int	%d	
	long	%ld	
• **Fractions**:	float	%f	%.nf
	double	%lf	%.nlf
• **ASCII**:	char	%c	

Num. of digits to the right of the decimal point that will be printed out.

Note: NO String datatype in C! Two alternatives using char.

String %s

DREW

Standard I/O in C: *printf()*

(Major) **Data types and variable declaration**

• **Integers**:	int	signed, 2's C	4 bytes (-2^{31}~$2^{31}-1$)
	long	signed 2's C	8 bytes
• **Fractions**:	float	signed, 2's C	4 bytes
	double	signed, 2's C	8 bytes
• **ASCII**:	char	unsigned	1 byte (0~255)

Standard I/O in C: *printf()*

Printing variables

```
int Location = 0x3000;
int Instruction = 0b1001010011101010;
printf("Current PC: 0x%x or %d in decimal\n", Location, Location);
printf("Current Instruction: 0x%x in hexadecimal\n", Instruction);

const float pi = 3.1415926535;          printf("pi = %.?f\n", pi);
```

Standard I/O in C: *printf()*

Printing variables

```
char mode_c = 'C';   char mode_d = 67;
char mode_h = 0x43;   char mode_hc = '\x43';
printf("Mode: %c\n", mode_?);
```

Standard I/O in C: *printf()*

Printing variables

```
char mode_c = 'C';   char mode_d = 67;
char mode_h = 0x43;   char mode_hc = '\x43';
char mode_b = 0b01000011;
printf("Mode: %c\n", mode_?);
```

- What about? (HW 7 available today)

```
char mode_c = (char) 'C'+1;   char mode_d = 67+1;
char mode_h = 0x43+1;   char mode_hc = (char) '\x43'+1;
char mode_b = 0b01000011+1;
printf("Mode: %c\n", mode_?);
```

Standard I/O in C: *scanf()*

- What about input? To scan from keyboard, we use *scanf()*.
- Uses the same formatting mechanism as printf().

- **Ex.**: To scan any number datatype or char:
  must add & (ampersand) before the variable!!

  scanf("%d", &num); *scanf("%c", &mode);*

- Again, has something to do with pointers.

Standard I/O in C: *scanf()*

- When scanning strings, a bit different. And need to BE CAREFUL!

- **Ex.**: To scan a string: no need for &.

  scanf("%s", myName);

- **Note**: Recall, to declare a String, we used char array: *char myName[20]*.
- Here, must ensure input has at most (<=) 19 chars.
- Otherwise, overflow!! Will overwrite data in other places.

- Also, not allowing spaces. Will treat "Hello World" as two strings.

Standard I/O in C: *scanf()*

```
int num;   char ch;   char str[10];   char* str_p;

printf("Enter a number: ");   scanf("%d", &num);   printf("%d\n", num);

...   scanf("\n%c", &num);   ...   // BE CAREFUL HERE

...   scanf("%s", str);   ...

...   scanf("%s", str_p);   ...
```

DREW

File Operations in C

- Can read in from and write out to files using C commands.
- Our files are just sequences of ASCII chars, or *text/data streams*.

- To deal with file content, we need to specify the file to read from/write to.

- First, define a *file pointer* that is the file stream, as

  FILE ptrName;*

- Then, need to associate the pointer with the actual file, using *fopen()* function.

File Operations in C

- Then, need to associate the pointer with the actual file, using *fopen()* function.

  FILE fopen(char* filename, const char* mode);*

  - Returns a file pointer.
  - First arg.: file name, that can be a path (in Linux style).
  - Second arg.: operation mode, what action(s) to take on the file.
    - "r": read from file only. [File MUST exist.]
    - "w": write to file only (overwrite from the beginning). [Creates if file does not exist.]
    - "a": write to file only (append to the end). [Creates if file does not exist.]

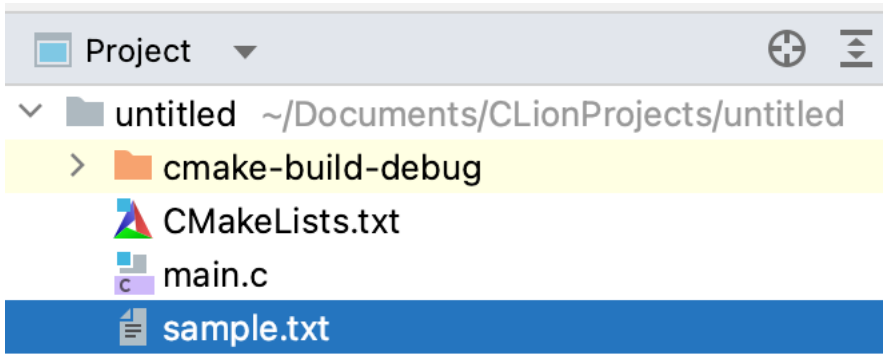
File Operations in C

Ex.: In CLion, the default path for an I/O file is under project folder.

- Say, want file *sample.txt* to have the same path as *main.c*:

```
FILE* filein = fopen("../sample.txt", "r");

// Check if successful before proceed
if (filein == NULL) {
        printf("File reading error!\n");
        return 1;
} ... ...
fclose(filein);        // Close the file after done r/w to it.
```



DREW

Functions to Read and Write – Formatted

- Similar to *printf()* and *scanf()* that deal with "formatted" I/O to console, to deal with formatted I/O to file, we use
  - *fscanf(char* filename, "format", [char* varName, ...])*
  - *fprintf(char* filename, "format", [char* varName, ...])*

where,

%d: int	%ld: long
%f: floating-point	%lf: double
%b: unsigned binary	%x: unsigned hexadecimal
%c: ASCII char	%s: ASCII string

Functions to Read and Write – Formatted

- Ex.:

```
FILE* filein = fopen("../sample.txt", "r");
char ch;
int num;
char* str;

fscanf(filein, "%s", str);          // read a string until a whitespace or
                                    // newline is hit

// for non-pointer datatypes, need the ampersand!
fscanf(filein, "%c", &ch);          // read the current character
fscanf(filein, "%d", &num);         // read next number until whitespace
```

Functions to Read and Write – Formatted

- **Ex.**: If the input file itself is formatted as:

Name	ID#	ResidenceHall	Number
Alice	123456	Asbury	212-111-0000
Bob	765432	Baldwin	703-222-4567

Then, can read as *fscanf(filein, "%s %d %s %d-%d-%d", name, &id, residence, &cell_area, &cell_3, &cell_4);*

And to add more record entries:

FILE fileout = fopen("../sample.txt", "a");*

fprintf(fileout, "\n%s\t%d\t%s\t%d-%d-%d", ...);

Functions to Read and Write – Formatted

- Additional formatting for special chars:

\n: newline \t: tab

\\: back slash

\': single quotation \": double quotation

Functions to Read and Write – Formatted

- **Ex.**: To keep reading, word by word

```
char str[20];          // say each word has a max length of 20 letters

while (fscanf(filein, "%s", str) != EOF) {
        // this while loop will keep reading the next string (until space)
        // from the stream automatically until End of File is reached

        fprintf(fileout, "%s\n", str);
}
```

Functions to Read and Write – Formatted

- **Ex.**: To keep reading, char by char

```
char ch;

while (fscanf(filein, "%c", &ch) != EOF) {
        fprintf(fileout, "%c", str);
        // Note: space and newline are all chars!
}
```

Functions to Read and Write – Not Formatted

- How about we want to read in a whole line; i.e., don't stop upon whitespaces?
- Can be done with *fscanf()*, but bit complicated.

- Alternatively, can use *fgets(char* varName, int scan_len, FILE* filename)* that is NOT formatted.
  - Scan until newline is detected (include the \n in var.); i.e., scanning line by line.
  - **Ex.**:

```
#define LEN 20
… …
char str[LEN];                    // Note: compare with NULL!!
while (fgets(str, LEN, filein) != NULL) { … }
```

Functions to Read and Write – Not Formatted

- Just as *fscanf()* is paired with *fprintf()*,
  fgets() is paired with *fputc()* and fputs() for char/string resp.

- *fputc()* and fputs()
  - Similar effects without format.
  - *fputc(char ch, FILE* filename)*
  - *fputs(char* str, FILE* filename)*